

Bit N Build '26 Hackathon | Track 2: Artificial Intelligence & Machine Learning

KAREN'S EAR: AI Emergency Dispatch Assistant

System Architecture, Algorithmic Methodology, and Operational Blueprint

Team Members: Preetika (Priority & ML), Rishav (FastAPI & Dashboard), Ojas (NLP), Aditya (Datasets) | **Stack:** Python, FastAPI, React

Executive Summary & Core Operational Paradigm

Karen's Ear is an intelligent, edge-resilient emergency triage assistant designed to cut through the noise, fragmentation, and volume of civic distress communications. Built around the six-phase operational paradigm:

Detect → Understand → Corroborate → Predict → Rank → Dispatch

The system pairs tri-state relevance filtering (High / Uncertain / Low) and a lexical safety net with multi-attribute priority regression ($\mathcal{P} \in [0.0, 1.0]$) to transform incoming text streams into an actionable, prioritized picture of emergency need.

1. Problem Definition and Operational Context

1.1 The Challenge: Information Deluge in Emergency Dispatch

In high-stress urban crises, 911 dispatch centers and digital communication feeds face catastrophic information overload:

- Bursting Volume & Concurrency:** Emergency distress traffic spikes by orders of magnitude within minutes of an incident, causing cognitive bottlenecks.
- Linguistic Fragmentation:** Distress messages arrive as panicked, incomplete sentence fragments (e.g., *"5th ave fire big smoke help stuck"*).
- Redundancy & Ambient Noise:** Multiple callers report the identical incident using disparate phrasing, while non-emergency chatter dilutes urgent signals.
- Semantic Ambiguity:** Critical hazard markers (*"something blew up"*) must be distinguished from benign mentions (drills, film shoots, fireworks).

1.2 Official Hackathon Problem Statement

Track 2: "Karen's Ear" — AI Emergency Dispatch Assistant

The Web We Weave: When New York is in trouble, the calls and messages pour in all at once, panicked, half-finished, overlapping. *"There's smoke on 5th."* *"Someone's stuck under the rubble."* *"The lizard-thing is heading downtown."* Buried in that flood of words is exactly the information Spider-Man needs: what's happening, where, and how urgent. But no human can read it all fast enough, and by the time the important message is spotted, it's too late. Karen, Peter's AI assistant, needs to cut through the chaos and tell him where to swing first.

Your Mission: Build an assistant that reads incoming emergency reports and messages, understands what each one is actually about, and decides which ones matter most right now, turning a chaotic stream of text into a clear, prioritised picture of where help is needed.

1.3 System Scope and Human-in-the-Loop Philosophy

Karen's Ear functions strictly as an **intelligence amplification and decision-support layer**. It does *not* replace human first-responder judgment. Rather, it structures unstructured dispatches, clusters corroborating eyewitnesses, computes explainable priority scores, and highlights critical life-safety hazards, enabling sub-second tactical triage.

2. Canonical Data Schema and Contract

To eliminate schema drift across microservices, Karen's Ear establishes a unified, strictly typed incident contract (Table 1).

Table 1: Karen’s Ear Canonical Incident Schema Contract

Field	Data Type	Permitted Enum	Domain / Operational Function
report_id	String	"R001", increment	Unique deterministic identifier for deduplication and audit tracking.
text	String	Arbitrary distress string	Raw, unmodified text as transmitted by citizen or emergency caller.
incident_type	Categorical	fire, explosion, accident, medical, collapse, flood, crime, missing, unknown	Standardized taxonomy classifying the primary hazardous event.
location	String / Null	Named landmark or null	Geocoded physical entity (e.g., "Times Square"). Null if unspecified.
severity	Categorical	low, medium, high, critical	Estimated threat to life safety, structural collapse, or physical injury.
actionability	Categorical	low, medium, high	Extent to which actionable dispatch details (e.g., trapped victims) exist.
credibility	Float	[0.0000, 1.0000] (Default 0.75)	Statistical confidence score based on message clarity and corroboration.
priority	Float	[0.0000, 1.0000]	Final triaged score determining position in dispatcher’s deployment queue.
status	Categorical	pending, dispatched, resolved	Operational lifecycle status of the emergency report.

Anti-Hallucination Directives: In emergency triage, hallucinated addresses or hazard types create dangerous real-world misallocations. If a location is not explicitly referenced in the text, `location` → `null`. If the hazard category cannot be ascertained, `incident_type` → `unknown`. Models must never guess missing factual information.

3. System Architecture and Pipeline

3.1 Architectural Blueprint

The pipeline enforces a decoupled, sequential architecture where each stage has a single verifiable responsibility (Figure 1).

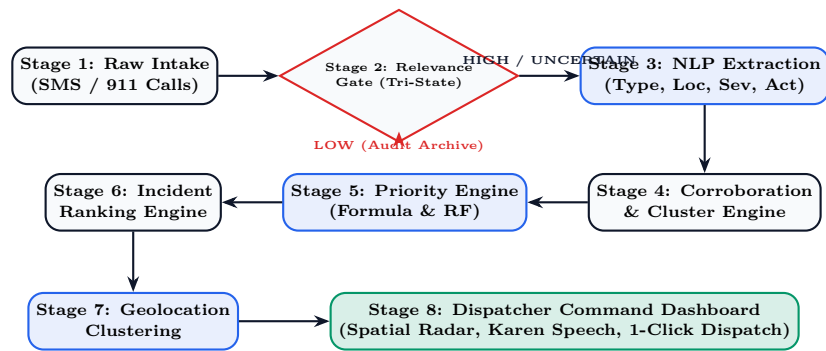


Figure 1: End-to-End Karen’s Ear Architectural Pipeline

3.2 Decoupling Relevance Gate from Priority Ranking

Relevance answers: “Does this message describe a real emergency?” Priority answers: “Where must first responders deploy first?” Conflating these dimensions causes dangerous operational confusion. A report saying “a cat is stuck on a garden fence” is 100% emergency-relevant, but has low priority (≈ 0.20). Conversely, a vague fragment “bomb subway” carries low grammatical certainty, but demands maximum priority vigilance (≈ 0.95). Decoupling

guarantees that low-urgency events are not discarded as noise, while ambiguous high-threat events are immediately flagged.

3.3 The 8-Stage Operational Flow

The Karen's Ear data pipeline executes eight disciplined transformations:

1. **Raw Report Intake:** Accepts multi-channel distress signals via asynchronous REST endpoints, generates unique identifiers, and normalizes encodings.
2. **Tri-State Relevance Gate:** Evaluates reports using TF-IDF and multinomial Logistic Regression into HIGH, UNCERTAIN, or LOW. Critical emergency markers are rescued via the lexical safety net.
3. **Incident Extraction:** Normalizes text into standard hazard categories (**fire**, **explosion**, **collapse**, etc.), extracts landmarks, and assesses severity and actionability.
4. **Corroboration Engine:** Groups concurrent reports referencing identical events using string distance and spatiotemporal clustering, dynamically incrementing credibility ($\mathcal{C}$) based on cluster size N .
5. **Priority Engine:** Synthesizes severity, actionability, credibility, and corroboration bonuses into a continuous, clamped priority score $\mathcal{P} \in [0.0, 1.0]$.
6. **Incident Ranking:** Maintains an active priority queue sorted strictly descending by $\mathcal{P}$ to ensure highest-urgency events dominate dispatcher visibility.
7. **Geolocation Aggregation:** Maps incidents into spatial sectors (e.g., Midtown, Downtown) to prevent deploying redundant apparatus to adjacent city blocks.
8. **Dispatcher Telemetry Dashboard:** Delivers real-time incident queues, spatial coordinates, Karen voice alerts, and one-click dispatch controls to operators.

4. Machine Learning and Algorithmic Methodology

4.1 Tri-State Relevance Classification

Binary classification is unacceptably brittle in civic safety: fragmented reports are easily dropped as false negatives. Karen's Ear introduces a **Tri-State Architecture**:

- **HIGH:** Unambiguous emergency markers with actionable details (e.g., *"Explosion at 5th Ave, people trapped"*).
- **UNCERTAIN:** Ambiguous, fragmented, or unverified reports containing emergency-adjacent signals. Passed forward with review flags.
- **LOW:** Verified benign chatter, spam, or explicit non-emergency statements (e.g., *"Did you see the explosion in the movie?"*).

Feature extraction uses **TF-IDF** with unigrams and bigrams ($n \in \{1, 2\}$) paired with a multinomial **Logistic Regression** classifier:

$$P(y = k | \mathbf{x}) = \frac{\exp(\mathbf{w}_k^T \phi(\mathbf{x}) + b_k)}{\sum_{j=1}^K \exp(\mathbf{w}_j^T \phi(\mathbf{x}) + b_j)}, \quad k \in \{\text{HIGH}, \text{UNCERTAIN}, \text{LOW}\} \quad (1)$$

4.2 Context-Aware Emergency Signal Safety Net

Because statistical models can under-score out-of-vocabulary emergency phrasing, Karen's Ear wraps the ML layer with a deterministic **Safety Net**:

- **Trigger Lexicon:** Scans for critical root stems: **explosion**, **fire**, **collapse**, **trapped**, **shooter**, **gunshot**, **flood**, **rubble**, **bleeding**.
- **Context Exclusion:** Checks for non-emergency qualifiers: **drill**, **movie**, **film**, **rehearsal**, **documentary**, **fireworks**, **museum**.
- **Safety Override:** If a trigger is detected without an exclusion context, and the ML model predicted **LOW**, the classification is automatically escalated to **UNCERTAIN**. It is never silently dropped.

4.3 Priority Engine Formulation

The Priority Engine computes a continuous priority metric $\mathcal{P} \in [0.0, 1.0]$. The deterministic core operates on four structured, normalized inputs (Table 2).

Table 2: Priority Engine Feature Weights and Numerical Encoding

Dimension	Categorical / Raw Input	Normalized Value	Formula Weight
Severity (S)	critical (1.00), high (0.75), medium (0.50), low (0.25)	$S \in [0.25, 1.00]$	$w_s = 0.45$
Actionability (A)	high (1.00), medium (0.60), low (0.20)	$A \in [0.20, 1.00]$	$w_a = 0.35$
Credibility (C)	Signal Confidence / Clustering	$C \in [0.00, 1.00]$	$w_c = 0.20$
Corroboration Bonus (Δ_k)	Incident Cluster Size N	$\min(0.05, (N - 1) \times 0.015)$	Additive Boost

The synthesized priority score $\mathcal{P}$ is defined mathematically by:

$$\mathcal{P}_{\text{raw}} = (0.45 \cdot S) + (0.35 \cdot A) + (0.20 \cdot C) + \min(0.05, (N - 1) \times 0.015) \quad (2)$$

$$\mathcal{P} = \text{round}\left(\max(0.0, \min(1.0, \mathcal{P}_{\text{raw}})), 4\right) \quad (3)$$

4.4 Explainability Engine and Automated Recommendations

In critical civic infrastructure, “black-box” scores are hazardous. Karen's Ear generates a real-time explainability payload for every incident, decomposing the score into its constituent drivers and providing an automated operational recommendation:

$$\text{Recommendation}(\mathcal{P}) = \begin{cases} \text{IMMEDIATE_DISPATCH}, & \text{if } \mathcal{P} \geq 0.80 \\ \text{HIGH_PRIORITY_STANDBY}, & \text{if } 0.60 \leq \mathcal{P} < 0.80 \\ \text{MONITOR}, & \text{if } \mathcal{P} < 0.60 \end{cases} \quad (4)$$

4.5 Random Forest Priority Regression Baseline

To discover non-linear feature interactions as training datasets expand, an ensemble Random Forest Regressor is trained on historical dispatch outcomes:

$$\hat{\mathcal{P}}_{\text{RF}}(\mathbf{x}) = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x}), \quad B = 100 \text{ estimators over } (S, A, C, \text{Corroboration}) \quad (5)$$

The ensemble baseline exhibits high stability against noisy outlier labels while producing a smooth continuous ranking curve.

5. System Implementation and Architecture

The system implementation is partitioned into modular, highly cohesive micro-layers:

- **Data Modeling (src/schema.py):** Validates inputs against Pydantic models with strict typing and numeric bounds.
- **Priority Core (src/priority/engine.py):** Implements deterministic feature weighting, corroboration bonuses, and explainability.
- **Relevance Pipeline (src/relevance/):** Manages TF-IDF vectorization, multiclass logistic regression, and the safety net rule engine.
- **API Backend (src/api/main.py):** High-performance asynchronous FastAPI REST endpoints for report ingestion and dispatch updates.
- **Command Dashboard (frontend/src/App.jsx):** React and Tailwind CSS console featuring New York landmark simulation, real-time audio alerts, and spatial radar.

5.1 Production Technology Stack

The production system leverages lightweight, high-performance open-source tools:

- **Core Runtime & Backend:** Python 3.10+ with **FastAPI** (ASGI/Starlette) for sub-10ms asynchronous request handling and automated OpenAPI documentation.
- **Data Modeling & Validation:** **Pydantic v2** enforcing strict type boundaries and runtime payload validation.
- **Machine Learning & Numerics:** **Scikit-Learn** (Logistic Regression, TF-IDF Vectorizer, Random Forest), **NumPy**, and **Pandas** for CPU-efficient, edge-friendly execution without GPU dependencies.
- **Frontend Telemetry:** **React 18** and **Vite** styled with **Tailwind CSS** for a high-contrast dark-mode tactical command console, paired with the browser **Web Speech API** for Karen voice alerts.

5.2 FastAPI REST Service Specifications

The backend provides sub-millisecond REST endpoints for ingestion and dispatch telemetry:

- **POST /ingest:** Ingests citizen distress text, auto-infers missing metadata, calculates priority score $\mathcal{P}$, and enqueues the record.
- **GET /reports:** Returns all stored emergency incidents sorted strictly in descending order of priority score $\mathcal{P}$.
- **PATCH /api/reports/{id}/dispatch:** Updates dispatch status and assigns a tactical response unit (e.g., "Engine 5").
- **GET /api/stats:** Returns aggregate analytics: total reports, critical hazard counts, and incident type breakdowns.
- **GET /health:** Liveness probe reporting operational health and active in-memory incident volume.

6. Evaluation Strategy and Verification Framework

6.1 Relevance Classification Evaluation: Why Macro Metrics Matter

In emergency dispatch datasets, class distribution is inherently skewed: routine traffic or benign chatter outnumbers structural collapses. Relying on global **Accuracy** creates a perilous illusion of performance:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{Total Reports}} \quad (6)$$

A naive model predicting 100% LOW in a dataset containing 95% non-emergencies would achieve 95% accuracy while missing every life-threatening event. Therefore, Karen's Ear mandatorily evaluates on **Macro-Averaged Precision, Recall, and F_1 -Score**:

$$\text{Precision}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{\text{TP}_k}{\text{TP}_k + \text{FP}_k}, \quad \text{Recall}_{\text{macro}} = \frac{1}{K} \sum_{k=1}^K \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k} \quad (7)$$

$$F_{1,\text{macro}} = \frac{2 \times \text{Precision}_{\text{macro}} \times \text{Recall}_{\text{macro}}}{\text{Precision}_{\text{macro}} + \text{Recall}_{\text{macro}}} \quad (8)$$

Evaluating across $K \in \{\text{HIGH}, \text{UNCERTAIN}, \text{LOW}\}$ guarantees that minority life-safety classes receive equal evaluation weight.

6.2 Priority Regression Metrics

Continuous priority regression is evaluated across three complementary metrics:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2}, \quad R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2} \quad (9)$$

6.3 Automated Unit Testing & Validation Suite

The codebase is validated via comprehensive automated unit tests in `tests/test_priority.py` and `tests/test_api.py`. Tests verify that: (1) Critical severity incidents with high actionability achieve $\mathcal{P} \geq 0.90$; (2) Multiple corroborating callers boost priority dynamically; (3) Scores strictly clamp within $[0.0, 1.0]$; and (4) FastAPI endpoints maintain 100% schema compliance.

7. Architectural Decisions and Rationale Matrix

Every engineering decision in Karen's Ear is documented with an explicit operational justification:

- **Why three relevance levels?** A binary gate is brittle. Fragmented reports that contain genuine emergency signals but lack sufficient context would be discarded. **UNCERTAIN** provides a safe review buffer.
- **Why TF-IDF Vectorization?** TF-IDF is lightweight, CPU-efficient, deterministic, provides sub-millisecond inference latencies, and operates without expensive GPU infrastructure.
- **Why Bigrams ($n = 2$)?** Emergency distress is expressed through short collocations rather than single words (e.g., "*shots fired*", "*water rising*"). Bigrams capture imperative intent.
- **Why an Emergency Safety Net?** Statistical models fail on out-of-distribution wording. The lexical safety layer guarantees critical terms ("*something blew up*") are never dropped due to vocabulary sparsity.
- **Why not force all keywords to HIGH?** High-consequence words appear in benign contexts (drills, movies, news). Blindly forcing HIGH causes alert fatigue. The safety net elevates to **UNCERTAIN**.
- **Why Random Forest for Priority?** The priority feature space is compact (≤ 6 dimensions) with non-linear interactions. Random Forest models non-linear thresholds cleanly without overfitting.
- **Why decouple Relevance from Priority?** Relevance addresses evidentiary signal presence; Priority orders incident urgency. Conflating them prevents independent auditing of classification versus dispatch order.
- **Why not use raw Location as Priority?** A prominent landmark (e.g., Times Square) must not receive an artificial urgency boost over a fatal residential accident in Queens. Location is reserved for spatial grouping and routing.

8. Team Integration, Operational Limitations, and Future Roadmap

8.1 Team Responsibilities and Module Ownership

Karen's Ear was developed by a specialized engineering team during Bit N Build '26:

- **Preetika:** Emergency priority and ranking model; relevance architecture and priority integration.
- **Ojas:** NLP incident classification, structured entity extraction, spatiotemporal corroboration, and credibility scoring.
- **Aditya:** Dataset preparation, synthetic augmentation, stratified splitting, and data curation pipelines.
- **Rishav:** FastAPI backend architecture, React command dashboard integration, and Web Speech API synthesis.

8.2 Current Prototype Limitations & Engineering Roadmap

- **Dataset Generalization:** Models were trained on representative synthetic vectors; calibration on municipal dispatch logs is pending.
- **Keyword Lexicon:** The safety net uses curated keyword lists requiring ongoing expansion for localized dialect idioms and multilingual slang.
- **Provenance Verification:** Credibility currently evaluates clustering density; production deployment requires caller identity and SIM-level reputation.
- **Multi-Vehicle Routing:** The system ranks incidents individually but does not yet perform multi-vehicle routing optimization (Vehicle Routing Problem).

Future Milestones: (1) Fine-tune quantized edge DistilBERT models; (2) Implement isotonic probability calibration; (3) Deploy spatiotemporal DBSCAN clustering; (4) Integrate Google OR-Tools for multi-vehicle dynamic fleet routing; (5) Cryptographic tamper-evident audit logging for operator overrides.

8.3 Conclusion

Karen's Ear re-imagines emergency response intelligence by cutting through the chaotic fog of civic disasters. By enforcing a disciplined multi-stage paradigm—*Detect* → *Understand* → *Corroborate* → *Predict* → *Rank* → *Dispatch*—the system eliminates information paralysis while preserving human dispatcher authority. Designed for edge resilience, transparency, and sub-second triage, Karen's Ear fulfills the Bit N Build '26 mission: giving emergency responders the ultimate "Spider-Sense" to know exactly where help is needed first.